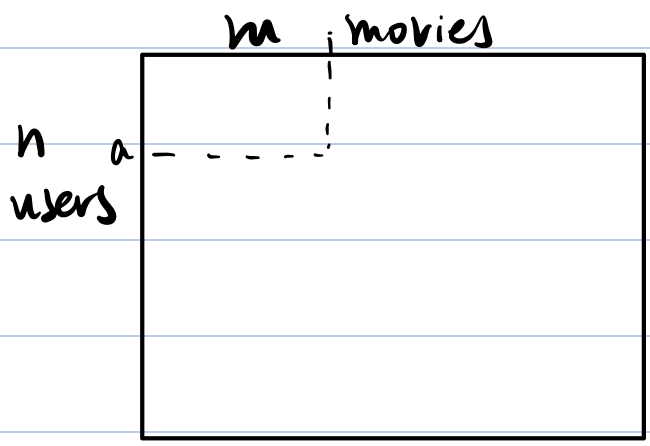
Recommender System

1. problem definition
2. k-nearest Neighbor algorithm
3. matrix factorization or collaborative filtering.

Problem definition

ranking matrix



predict the opinion of the user she or he hasn't seen.

k-nearest-neighbors

k: The size of the advisory pool.

V1: $\hat{y}_{ai} = \frac{1}{k} \sum_{b \in KNN(a,i)} y_{bi}$

KNN (a,i) : The nearest k neighbors of a who watch movie i

V2: $\hat{y} = \dfrac{\sum_{b \in KNN(a,i)} sim(a,b)\, y_{bi}}{\sum_{b \in KNN(a,i)} sim(a,b)}$

2. KNN 算法的工作原理

(1) 分类问题（KNN for Classification）

对于分类问题，KNN 的工作流程如下：

1. **选择参数 K**：选择一个正整数 K，表示在做出预测时，考虑多少个最近邻。
2. **计算距离**：给定一个新的测试样本，算法计算该测试样本与训练集中每个样本的距离，常用的距离度量方法包括：
   - **欧几里得距离（Euclidean Distance）**：
   $$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$
   - **曼哈顿距离（Manhattan Distance）**：
   $$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$
   - **闵可夫斯基距离（Minkowski Distance）** 等。
3. **选择 K 个最近邻**：根据计算的距离选择最近的 K 个训练样本。
4. **投票决定类别**：对 K 个最近邻进行"投票"，测试样本的类别由这 K 个邻居的多数类别决定。如果是二分类问题，则通常选择类别出现次数最多的那个类别作为测试样本的预测类别。
5. **返回预测结果**：输出预测的类别标签。

(2) 回归问题（KNN for Regression）

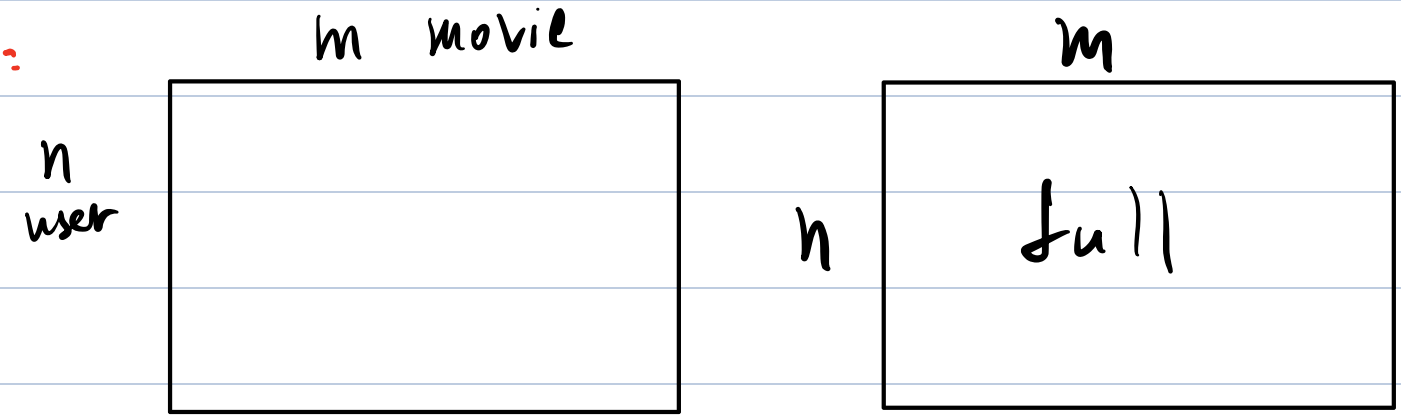
对于回归问题，KNN 的步骤类似，但预测方法不同。对于给定的测试样本，KNN 会选择 K 个最近邻，并对这些邻居的目标值（即标签值）进行平均或加权平均，作为预测结果：

1. **选择 K 个最近邻**：计算测试样本与训练样本的距离，选择最近的 K 个训练样本。
2. **计算预测值**：将 K 个最近邻的标签值取平均（或加权平均，权重通常与距离成反比）：
   $$\hat{y} = \frac{1}{K} \sum_{i=1}^{K} y_i$$
   或者使用加权平均：
   $$\hat{y} = \frac{\sum_{i=1}^{K} \frac{y_i}{d_i}}{\sum_{i=1}^{K} \frac{1}{d_i}}$$
   其中 d_i 是第 i 个邻居与测试样本之间的距离。

Collaborative Filtering

Collaborating Filtering

goal:



Y (given) X (output) (prediction)

$$J(x) = \sum_{(a,i)\in D} (|Y_{a,i} - X_{a,i}|)^2 / 2 + \frac{\lambda}{2} \sum_{(a,i)\in X} |X_{a,i}|^2$$

$$D = \{(a,i) \mid Y_{a,i} \text{ is given}\}$$

因为每个电影之间的评分没有联系

=> 可以分别预测

使 $J(x)$ 最小

Assume

① $(a,i) \in D$: $\frac{dJ(x_{a,i})}{dx_{a,i}} = 0 \Rightarrow x_{a,i} = \frac{Y_{a,i}}{1+\lambda}$

② $(a,i) \notin D$: $\frac{dJ(x_{a,i})}{dx_{a,i}} = 0 \Rightarrow x_{a,i} = 0$

$$\frac{dJ(x_{a,i})}{dx_{a,i}} = -|Y_{a,i} - x_{a,i}| + \lambda |x_{a,i}|$$

Matrix Factorization

Assumption : X is low rank

在线性代数中，**低秩 (Low Rank)** 矩阵是指秩（Rank）远小于其行数或列数的矩阵。这里的秩是指矩阵的**秩 (Rank)**，即矩阵中**线性无关行向量或列向量的最大数量**。

详细解释：

1. **矩阵的秩 (Rank)**：
   - 矩阵的秩表示矩阵中**非零的奇异值的个数**，或者说它是矩阵的**最大线性无关行或列的数量**。
   - 例如，一个 $m \times n$ 的矩阵，秩 $r \leq \min(m, n)$。
2. **低秩矩阵 (Low Rank Matrix)**：
   - 若矩阵 A 的秩 r 远小于其维度 m 和 n，则称 A 是低秩矩阵。
   - 通常，低秩矩阵的秩 r 比 $\min(m, n)$ 小得多。例如：

$$r \ll \min(m, n)$$

   - 举例：一个 100×100 矩阵如果秩为 2，就可以称为低秩矩阵。

直观理解：

- 低秩矩阵通常表示数据的冗余性较高，可以被更少的线性独立向量表示。
- 如果一个矩阵是低秩的，则可以用更小的矩阵或更少的向量来近似描述原始矩阵的信息。

Tank 1 :

$$\begin{array}{cc} & m \\ n & \begin{bmatrix} 1 & 2 & 3 \\ 5 & 10 & 15 \end{bmatrix} \end{array} = \begin{bmatrix} 1 \\ 5 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

$= \cdots\cdots$

① $O(n \cdot m) \Rightarrow O(n+m)$

$X = U \cdot V^T$ (Matrix X)

$X_{ai} = U_a \cdot V_i$ (rank 1)

U 表示的是 user 的数据

V 表示的是 movie 的数据

$$J(u,v) = \sum_{(a,i) \in D} (y_{ai} - u_a v_i)^2 + \frac{\lambda}{2} \sum_{a=1}^{n} u_a^2 + \frac{\lambda}{2} \sum_{i=1}^{m} v_a^2$$

问题变成了 $X_{output} = U \cdot V^T$
找出 U 和 V

Collaborative Filtering with Matrix Factorization（矩阵分解协同过滤）

1. 什么是 Collaborative Filtering（协同过滤）？

协同过滤是一种推荐系统的算法，主要基于用户的历史行为和相似用户的行为来进行推荐。其核心思想是：**相似的用户有相似的兴趣，用户会喜欢相似用户喜欢的项目。**

- **用户-物品矩阵**（User-Item Matrix）：用来记录用户对物品的评分。该矩阵通常很稀疏，因为大部分用户只对少量物品进行评分。

2. Matrix Factorization（矩阵分解）的基本概念

矩阵分解方法是协同过滤的一种重要技术，它可以将用户-物品评分矩阵 R 分解为两个低秩矩阵的乘积，从而发现用户和物品的潜在特征。

假设 R 是用户对物品的评分矩阵：

- $R_{i,j}$ 表示用户 i 对物品 j 的评分。
- 由于用户并没有对所有物品进行评分，因此矩阵 R 中存在许多**空值**。

通过矩阵分解，将 R 分解为两个矩阵的乘积：

$$R \approx U \times V^T$$

- $U \in \mathbb{R}^{m \times k}$：用户特征矩阵，$m$ 是用户数量，k 是潜在特征的数量。
- $V \in \mathbb{R}^{n \times k}$：物品特征矩阵，$n$ 是物品数量，k 是潜在特征的数量。
- k 通常远小于 m 和 n，因此实现了**低秩分解**。

3. Collaborative Filtering with Matrix Factorization 的核心思想

通过矩阵分解，我们可以用**用户和物品的隐含特征向量**来表示评分：

- 用户 i 被表示为向量 u_i。
- 物品 j 被表示为向量 v_j。
- 用户对物品的评分可以用两者的**内积**来估计：

$$R_{i,j} \approx u_i \cdot v_j^T$$

即用户 i 对物品 j 的预测评分是 u_i 和 v_j 的内积。

通常使用**梯度下降法**来优化参数 U 和 V：

- 不断更新 u_i 和 v_j 以减少损失函数值。

e.g.

Vectors: $u_1, u_2, v_1 \ldots$

$$Y = \begin{bmatrix} 5 & ? & 7 \\ 1 & 2 & ? \end{bmatrix}, \quad u = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix}, \quad v = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

goal: minimize $J(u,v)$

① initialize $v = \begin{pmatrix} 2 \\ 7 \\ 8 \end{pmatrix}$ (randomly)

② compute uv^T

$$uv^T = \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} \cdot \begin{pmatrix} 2 & 7 & 8 \end{pmatrix}$$

$$= \begin{pmatrix} 2u_1 & 7u_1 & 8u_1 \\ 2u_2 & 7u_2 & 8u_2 \end{pmatrix}$$

③ For u_1

$$\left(\frac{(5-2u_1)^2}{2} + \frac{(7-8u_1)^2}{2} + \frac{\lambda}{2} u_1^2 \right)'$$

$$= -2(5-2u_1) - 8(7-8u_1) + \lambda \cdot u_1$$

$$= 4u_1 - 10 + 64u_1 - 56 + \lambda \cdot u_1$$

$$= (68+\lambda) u_1 - 66$$

$$u_1 = \frac{66}{68+\lambda}$$

For $u_2 =$ .....

Finally get $u_2 = \frac{16}{\lambda+53}$

④ For $\lambda = 1$

$$u_1 = \frac{66}{69}, \quad u_2 = \frac{16}{54}$$

$$u = \begin{pmatrix} \frac{66}{69} \\ \frac{16}{54} \end{pmatrix}, \quad v = \begin{pmatrix} v_1 & v_2 & v_3 \end{pmatrix}$$

$$u \cdot v^T = \begin{pmatrix} \frac{66}{69} v_1 & \frac{66}{69} v_2 & \frac{66}{69} v_3 \\ \frac{16}{54} v_1 & \frac{16}{54} v_2 & \frac{16}{54} v_3 \end{pmatrix}$$

① compute v_1 and v_2 and v_3

② repeat until converge (guarantee local converge)